

- b. `initiatorICAC` (if present) as a `matter-certificate`
- c. `InitiatorEphKeyPair.publicKey`
- d. `Msg2.responderEphPubKey`

3. The initiator SHALL `generate a signature`:

```
TBSData3Signature = Crypto_Sign(
    message = sigma-3-tbsdata,
    privateKey = InitiatorNOKeyPair.privateKey
)
```

4. The initiator SHALL encode the following items as a `sigma-3-tbedata`:

- a. `initiatorNOC` as a `matter-certificate`. This encoding SHALL be byte-for-byte identical to the encoding in `sigma-3-tbsdata`.
- b. `initiatorICAC` (if present) as a `matter-certificate`. This encoding SHALL be byte-for-byte identical to the encoding in `sigma-3-tbsdata`.
- c. `TBSData3Signature`

5. The initiator SHALL `generate the S3K key`.

6. The initiator SHALL `generate the encrypted and integrity protected data`:

```
byte TBEData3_A[] = {}
byte TBEData3_Nonce[13] = /* "NCASE_Sigma3N" */
    {0x4e, 0x43, 0x41, 0x53, 0x45, 0x5f, 0x53, 0x69,
     0x67, 0x6d, 0x61, 0x33, 0x4e}

TBEData3Encrypted = Crypto_AEAD_GenerateEncrypt(
    K = S3K,
    P = TBEData3,
    A = TBEData3_A,
    N = TBEData3_Nonce
)
```

- 7. Any context-specific tags not listed in the above TLV schemas SHALL be reserved for future use, and SHALL be silently ignored if seen by a responder which cannot understand them.
- 8. The initiator SHALL send a message with `Secure Channel Protocol ID` and `Sigma3 Protocol Opcode` from Table 18, “`Secure Channel Protocol Opcodes`” whose payload is the TLV-encoded `Sigma3 Msg3 = { encrypted3 (1) = TBEData3Encrypted }` with an anonymous tag for the outermost struct.
- 9. The initiator SHALL generate the session encryption keys using the method described in Section 4.14.2.6.6, “`Session Encryption Keys`”.